BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 045 · Abstraction and Decomposition

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What makes a technology issue ethical?

Q2. Which technology does a USB flash drive use?
A. Solid-state
B. Optical
C. Magnetic
D. Volatile RAM

Q3. “Compression must increase every file's size.” Correct this. Explain why.

Q4. A website thumbnail may tolerate a small reduction in detail. “Lossy compression guarantees an identical original after decompression.” Correct this. What could happen?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Computational thinking — Techniques for representing and solving problems logically.

Decomposition — Breaking a complex problem into manageable smaller parts.

Abstraction — Focusing on relevant details and ignoring unnecessary details.

Subproblem — Smaller task forming part of a larger problem.

Relevant detail — Information needed for a solution.

Algorithmic thinking — Developing a logical sequence of steps to solve a problem.

New learning

Computational thinking: Computational thinking helps define a problem so a computer solution can be designed. OCR J277 specifies abstraction, decomposition and algorithmic thinking.

Decomposition: Split a large problem into manageable subproblems, such as input, calculations, checking and output. Each subproblem can be developed and tested more easily.

Abstraction: Identify essential details and ignore information not needed to solve the problem. A simplified map may show roads and stations but omit the colour of nearby buildings.

Use both together: First identify the required result; break the job into parts; retain information each part actually needs. Abstraction does not mean deleting important data.

Worked example

A club wants a program to calculate the cost of booking sports courts. Explain how decomposition and abstraction would help create the program. [4 marks]

1. Define the task: calculate and display a booking cost.

2. Decompose into input duration, get hourly price, calculate total and display it [1], making development manageable [1].

3. Abstract away irrelevant details such as court wall colour [1], keeping duration and rate needed to calculate cost [1].

4. Use specific examples from the booking problem.

We Do · Practise with guidance

Guided 1

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

A cinema needs an online ticket-pricing program. Explain two ways decomposition helps the programmer. [4 marks]

Worked method: Break it into input age/ticket type, calculate prices and output total [1], making each subproblem manageable [1]. Each component can be tested separately [1], helping locate errors [1].

Guided 2

A bus-route planner uses distances and stops but ignores building colours. Explain how abstraction helps. [4 marks]

Worked method: Abstraction keeps distances and stops needed for route calculation [1], so the model contains relevant features [1]. It ignores building colours [1] because that information does not affect the route [1].

Guided 3

A school lunch app totals several purchases. Explain how decomposition and abstraction work together. [4 marks]

Worked method: Decomposition splits the task into taking item prices, adding totals and displaying payment [1], simplifying work [1]. Abstraction keeps prices and quantities [1] while ignoring irrelevant details such as a pupil's favourite colour [1].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Code First · Predict → Write → Test → Improve

Write a line to show the cost of three storage cards.

```
unitPrice = 5
quantity = 3
```

write

Question 1

Define abstraction in computational thinking. [2 marks] Source: Original OCR J277-style teaching question; not an official OCR past paper.

Question 2

A weather app predicts whether to show an umbrella warning. Explain how decomposition and abstraction help design it. [4 marks] Source: Original OCR J277-style teaching question; not an official OCR past paper.

Question 3

Explain abstraction and decomposition using an example of a simple school timetable program. [6 marks] Source: Original OCR J277-style teaching question; not an official OCR past paper.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Exit Ticket · Answer before checking

Abstraction removes information that is irrelevant to the problem while keeping useful details.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. It concerns fairness, responsibility or what ought to be done, even if an action is legal.

Q2. A — Solid-state

Q3. Reducing file size can reduce storage requirements and transmission time.

Q4. Lossy compression removes information, so the exact original may be lost. A thumbnail can lose some detail; source code must be restored exactly.

I Do / We Do pages · Guided practice

Q1. Break it into input age/ticket type, calculate prices and output total [1], making each subproblem manageable [1]. Each component can be tested separately [1], helping locate errors [1].

Q2. Abstraction keeps distances and stops needed for route calculation [1], so the model contains relevant features [1]. It ignores building colours [1] because that information does not affect the route [1].

Q3. Decomposition splits the task into taking item prices, adding totals and displaying payment [1], simplifying work [1]. Abstraction keeps prices and quantities [1] while ignoring irrelevant details such as a pupil's favourite colour [1].

You Do pages · Code First

```
unitPrice = 5
quantity = 3
print(unitPrice * quantity)
```

Expected result: 15

You Do pages · Independent questions

Q1. Removing/ignoring irrelevant detail [1] while keeping information required to solve a problem [1].

Q2. Break the job into reading forecast, comparing rain probability and displaying advice [2]. Keep forecast/rain likelihood [1] while ignoring irrelevant information such as the user's phone case colour [1].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Q3. Decomposition splits the program into loading lesson data, finding a day and displaying the timetable [2], helping development/testing [1]. Abstraction keeps lesson time, room and subject [1] but ignores details not needed such as wall colour [1], reducing complexity without losing essential information [1].

Exit Ticket · Model answer

True

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: